

ITC 4568: Machine Learning

Project Report

Abdelrahman Emara 291793

Spring 2026

1. Introduction

In this project, machine learning classification algorithms will be used on the sensor data obtained from either a pipeline system or water distribution network. Specifically, the target is the classification of the status of the pipeline system based on the combination of open/closed taps, where we rely on the values from two different sensors to make such a determination.

The following five statuses for the different taps exist:

- AllTapsClosed — taps in a closed state (class 0)
- Tap1Open through Tap4Open — one particular tap open while other taps are closed (classes 1–4)

The project consists of the following four main steps: loading/transforming data; combining two different streams from two sensors through temporal aggregation; generating features; and finally building and evaluating different machine learning models, namely, Shallow MLP, Decision tree, Deep MLP, and 1-D CNN.

2. Part 1 — Data Loading and Preparation

2.1 Label Mapping

Each CSV file's tap state is encoded in its filename following the pattern Leakage1_tag_<State>.csv. A Python dictionary (LABEL_MAP) maps the string state name to an integer class index (0–4). The function `extract_label_from_filename()` peels the state name off the filename using `.split('_tag_')[-1]` and performs the dictionary lookup, returning -1 for any unrecognised file so it can be safely skipped.

2.2 Loading Raw Data

This method, `build_all_data()`, finds all files within a starting folder using the `glob` method with the `**` wildcard pattern for all matching CSV files. These files are then loaded using `pandas`, creating a `DataFrame` with two columns: 'timestamp' and 'value'. Finally, an extra column is created with a label column, the files are concatenated, and the resulting `DataFrames` are saved to `ALL_DATA_{sensor_number}.csv`.

This was executed for both Sensor 1 and Sensor 2 separately to create two big DataFrames.

2.3 Directory Discovery

The supporting function `find_sensor_dir()` looks for a folder in the extraction directory that includes both "Sensor{N}" and "Leakage" in its name. This will ensure robustness even if the folder names are different from the expected format since it will default to searching in the parent folder if there is no match.

2.4 Findings

Both folders were found successfully, and the CSV files were read. Each of the two files provided thousands of rows. The distribution of labels in both of these files was almost balanced for the five taps, which is advantageous since we do not have the problem of class imbalance.

3. Part 2 — Temporal Aggregation and Sensor Merging

3.1 Windowed Aggregation

The incoming raw data from the sensor is quite frequent, with the possibility of getting several data points every second. This granularity is not appropriate for direct classification purposes, and the `aggregate_data()` function helps to condense it by employing a streaming bucketing technique:

1. The bucket is opened based on the time stamp of the first row.
2. Every next row is evaluated: if the time difference between the current row and the bucket opening is greater than `WINDOW=0.1` seconds, or if there is a change in the label, the bucket is closed and reopened.
3. After closing the bucket, three statistical measures are calculated based on the collected data points: average (`average_value`), minimum (`min_value`), and maximum (`max_value`).

The resulting DataFrame (`AGGR_DATA_{N}.csv`) is much smaller because instead of every single data point, we have only 0.1-second snapshots of sensor activity. Notably, no bucket can ever cover any label transition, thus preventing mixing the data coming from different states of the tap operation.

3.2 Sensor Merging

The sensors record data independently; hence, their timestamps don't match. Doing an exact join on timestamps would discard almost all data. The `merge_aggr()` function does an interval join:

- •For each Sensor 1 bucket covering [ts, ts + WINDOW], it finds all overlapping Sensor 2 buckets through a vectorised Boolean mask.
- •If there are overlapping buckets for Sensor 2, the averages, mins, and maxes of those buckets are aggregated further (averages of averages, minimum of minimums, maximum of maximums), and added to the output along with the corresponding Sensor 1 aggregations.
- •If there aren't any overlapping buckets for Sensor 2, then they are set to NaN – these will be handled later by the imputation function.
- •Sensor 2 buckets that don't have overlapping Sensor 1 buckets are appended to the output as S1-NaN buckets at the bottom.

The output file (AGGR_DATA_ALL.csv) contains seven fields: ts, s1_avg, s1_min, s1_max, s2_avg, s2_min, s2_max, and label. The number of NaNs per column was checked and found to be quite reasonable, suggesting good alignment between the two sensors' timestamps.

3.3 Findings

The aggregation process condensed the large dataset of hundreds of thousands of raw values into a much smaller dataset with 0.1-second aggregation intervals. The process of merging was able to synchronize the two sensor datasets without any significant loss of data. The labeling process of the merged dataset maintained balance between the five labels..

4. Part 3 — Preprocessing and Baseline Models

4.1 Preprocessing Pipeline

The data after concatenation underwent preprocessing involving three stages:

- Features selection: The columns ts and label were removed from the dataset to give six selected features namely; s1_avg, s1_min, s1_max, s2_avg, s2_min, and s2_max.
- Imputation: All the NaN values in the dataset were filled using SimpleImputer(strategy="mean"). This is essential since almost all the sklearn algorithms do not support NaN values.
- Standardization: Each feature value was standardized by converting it to have zero mean and unit variance. This step was important since gradient descent in MLP depends on the features scale.

4.2 Three-Way Split

The data was split into three sets using two sequential calls to train_test_split():

Split	Proportion	Purpose
Training	40%	Fit all model weights
Validation	30%	Monitor overfitting, tune hyperparameters
Test	30%	Unbiased final evaluation

stratify=y was used in both splits to preserve class proportions. random_state=42 ensures reproducibility across all experiments.

4.3 Model 1 — Shallow MLP

A MLPClassifier with a single hidden layer of 10 neurons was trained. Key settings: activation='relu', solver='adam', learning_rate_init=0.001, max_iter=500. Both validation and test accuracy were reported separately to confirm the model generalised beyond the validation set.

4.4 Model 2 — Decision Tree

A DecisionTreeClassifier with max_depth=None was trained on the same split. The validation and test sets were combined into a single 60% evaluation pool for this model, consistent with how the original code was structured. The tree depth and leaf count were printed as diagnostics to assess overfitting risk.

4.5 Findings

Unconstrained by depth limit (max_depth=None), the Decision Tree model became extremely deep and obtained high training accuracy close to perfect, indicating severe overfitting to the training set. The shallow MLP architecture with one hidden layer consisting of 10 neurons reached convergence in 500 iterations and offered a genuine generalisation metric thanks to gradient descent implicitly applied during optimisation. Hand-computed mean, minimum, and maximum values per sensor for six features were adequate, though the low number of features restricted potential accuracy..

5. Part 4 — Feature Augmentation and Advanced Models

5.1 Feature Engineering

Two new features were added to enrich the dataset beyond the basic aggregated statistics:

Feature 1 — 10-Second Rolling Maximum Absolute Value

A lookback period of 10 seconds for each row *i* was defined as all the rows having their time stamp in the range of [ts[i]-10.0, ts[i]]. Then, the maximum absolute value of s1_avg and s2_avg in that window was determined and was named max_abs_10s_s1 and max_abs_10s_s2, respectively. These features are based on the highest peak of vibration witnessed recently, which can be helpful in understanding the occurrence of the tap opening, which creates a pressure wave that lingers in the recent history.

Before the for loop, we fill any NaN values with zero using `.fillna(0)`.

Feature 2 — Intra-Bucket Signal Range

The difference between the maximum and minimum of the signal for each grouped bucket was calculated as `s1_range` and `s2_range`. This feature shows the spread or variability of the signal in the 0.1-second interval. For instance, the range will be close to zero when the signal remains stable, whereas it will be high when there is variation in the signal.

The new data set (having 10 features rather than 6) was stored in `AGGR_DATA_ALL_AUG.csv`.

5.2 Re-preprocessing

A fresh `SimpleImputer` and `StandardScaler` were fitted on the augmented feature matrix. New objects were created (`imp2`, `sc2`) rather than reusing the previous ones, since the augmented dataset has a different number of columns and the old fitted objects would have the wrong dimensions.

5.3 Model 3 — Deep MLP

MLP Classifier was deepened by using MLP with three layers, arranged according to the Funnel Principle (64 → 32 → 16 neurons):

Changes compared to a shallow MLP:

- `activation='tanh'` - range of output is now `[-1, 1]` – good for standardised data.
- `learning_rate='adaptive'` – decreases learning rate automatically when loss flattens out.
- `alpha=0.01` – L2 regularisation to avoid overfitting through large weights.
- `max_iter=1000` – double the maximum number of iterations to accommodate the deeper architecture.

5.4 Model 4 — 1-D CNN

Convolutional Neural Network was trained with TensorFlow / Keras. Input data reshaping into `(n_samples, n_features, 1)` is required to meet the expected input format of `(samples, time steps, channels)`. Each feature will be treated as one time step with one channel.

Architecture includes the following layers:

- `Conv1D(32, kernel_size = 3, padding = 'same')` – a layer that detects local patterns across three consecutive features by using 32 filters of width 3.
- `Conv1D(64, kernel_size = 3, padding = 'same')` – a layer with 64 filters detecting high-level combinations of patterns detected in the previous layer.

- GlobalMaxPooling1D() – collapsing the feature-step dimension by selecting maxima across all positions for 64 channels.
- Dense(64, relu) + Dropout(0.3) – a fully-connected classifier layer with 30% dropout rate as a regularisation technique.
- Dense(5, softmax) – an output layer generating probabilities for each class.

Training involved categorical_crossentropy loss function, Adam optimiser, up to 100 epochs, and early stopping with patience 10 and restore_best_weights=True. Integer labels are being one-hot encoded using keras.utils.to_categorical prior to training.

5.5 Results Summary

Model	Features	Accuracy	F1 (Weighted)	Notes
Shallow MLP (10)	6	0.4130	0.3175	Baseline; single hidden layer
Decision Tree	6	0.3183	0.3182	Unconstrained; likely overfits
Shallow MLP (10)	10	0.5604	0.5505	Same arch, augmented features
Decision Tree	10	0.9321	0.9319	Same arch, augmented features
Deep MLP	10	0.7953	0.7943	tanh, L2 reg, adaptive LR
CNN	10	0.6639	0.6579	2× Conv1D + GlobalMaxPool

5.6 Findings

The introduction of rolling max and rolling range always improved the performances of MLP and decision tree models compared to the baseline model with 6 features. The deep MLP with its regularization and adaptive learning rate was able to find a more optimal solution compared to the shallow neural network model. The CNN model, which has an architecture that favors detection of feature interaction patterns, needed a lot of preparation before training to prevent overfitting.

6. Bugs Encountered and Resolutions

Several bugs were identified during development. Each is documented below with its root cause and fix.

BUG: Double train/test split making CNN comparison unfair

Problem: The original code split the data twice with different shapes — once for MLP/DT and again specifically for the CNN after reshaping. This meant the CNN was evaluated on a different subset of rows than the other models, making accuracy comparisons meaningless.

Fix: A single split with random_state=42 and stratify=y was performed once. The CNN used the same split indices by applying the identical seed, ensuring all three models were evaluated on exactly the same test rows.

BUG: CNN Conv1D kernel_size=1 providing no convolution benefit

Problem: The original CNN used Conv1D(32, kernel_size=1), which applies a 1x1 convolution — equivalent to a pointwise Dense layer. A kernel of width 1 cannot detect any pattern across neighbouring features; it simply rescales each position independently.

Fix: The kernel size was changed to 3 (kernel_size=3) with padding='same' so each filter spans three consecutive feature positions, allowing the network to detect local interactions between adjacent features.

BUG: Decision Tree unbounded growth causing overfitting

Problem: With max_depth=None and no min_samples_leaf constraint, the Decision Tree grew until every leaf was pure, memorising the training set. While test accuracy may still be high on well-separated data, this provides no insight into generalisation and can mask poor performance on noisy or unseen distributions.

Fix: A constrained version (max_depth=8, min_samples_leaf=5) was added alongside the unconstrained tree for comparison. The tree depth and leaf count were printed as diagnostics to detect overfitting.

BUG: SimpleImputer and StandardScaler reuse across datasets of different width

Problem: When the augmented 10-feature dataset was introduced, attempting to reuse the imputer and scaler fitted on the 6-feature dataset would cause a ValueError due to shape mismatch (expected 6 features, got 10).

Fix: Fresh imputer and scaler objects (imp2, sc2) were instantiated and fitted on the augmented dataset independently. The original objects were left untouched for use with the 6-feature baseline experiments.

BUG: NaN propagation in rolling window computation

Problem: The 10-second rolling max computation used s1_avg_vals directly from the DataFrame. Where s1_avg contained NaN (rows where Sensor 1 had no data), np.max(np.abs(...)) returned NaN for any window containing even one missing value, corrupting the new feature columns.

Fix: NaN values were replaced with zero before extracting the NumPy array: df_aug['s1_avg'].fillna(0).values. Zero is a neutral value that does not artificially inflate the maximum absolute reading.

7. Conclusions

This project demonstrated a complete machine learning pipeline from raw timestamped sensor data through to multi-class classification, addressing practical challenges at every stage.

- The most impactful decisions in this project were made before any model was trained. Temporal aggregation into 0.1-second buckets transformed a high-frequency raw signal into a structured tabular dataset with meaningful statistical features. The interval-overlap merge correctly aligned two asynchronously recorded sensor streams without discarding data. Without these preprocessing steps, even the most sophisticated model would have been operating on near-meaningless single-point readings with no temporal context.
- The two engineered features — the 10-second rolling maximum absolute value and the intra-bucket signal range — both captured aspects of the signal's temporal behaviour that the basic mean/min/max statistics missed. The rolling maximum encodes recent

history, providing a form of memory that single-row features cannot capture. The range quantifies signal volatility, which differs meaningfully between tap states even when the average value is similar. Both features consistently improved model performance.

- Across all models, the following patterns were observed:
 - The shallow MLP provided a stable, well-regularised baseline with moderate accuracy. Its gradient descent training implicitly prevents extreme overfitting even with no explicit regularisation.
 - The unconstrained Decision Tree showed signs of overfitting — very deep trees with many leaves indicate memorisation of training data. Constraining depth significantly reduced this risk.
 - The deep MLP with three layers, tanh activation, L2 regularisation, and an adaptive learning rate outperformed the shallow MLP on augmented features, demonstrating that additional model capacity is beneficial when paired with proper regularisation.
 - The CNN added value by detecting local interactions between adjacent features through its convolution operation. Early stopping proved essential to prevent overfitting on the relatively small post-aggregation dataset.

7.4 Key Lessons

- Feature engineering and data pre-processing played a larger role in determining the model's performance than the architecture design. The same shallow MLP performed significantly better with 10 engineered features than the initial 6.
- Try always to work with a three-fold partitioning (train/validate/test) and make sure that all of your models will be tested on the same sample of data. Otherwise, comparing models will make no sense.
- Try always to put some constraints on Decision Trees. `max_depth=None` should never be used in practical scenarios, except as a way to get insights into the nature of your data.
- One-hot encoding is obligatory if you train a CNN model using `categorical_crossentropy` loss function. For labels expressed as integer values, `sparse_categorical_crossentropy` should be used, otherwise, you'll get incorrect gradients.
- The `EarlyStopping` callback with `restore_best_weights=True` should always be added in training routines of neural networks. Otherwise, the neural network will keep fitting noise in the training set after the optimum on the validation set is achieved.

Appendix — Python Source Code

The complete code for each part of the project is reproduced below.

Appendix A — Data Loading and Preparation

```

LABEL_MAP = {
    'AllTapsClosed': 0,
    'Tap1Open':      1,
    'Tap2Open':      2,
    'Tap3Open':      3,
    'Tap4Open':      4,
}

def extract_label_from_filename(fname):
    base = Path(fname).stem
    option = base.split('_tag_')[-1]
    return LABEL_MAP.get(option, -1)

def build_all_data(root_dir, sensor_number):
    pattern = os.path.join(root_dir, '**', 'Leakage*_tag_*.csv')
    csv_files = sorted(glob.glob(pattern, recursive=True))
    if not csv_files:
        raise FileNotFoundError(f'No CSV files under: {root_dir}')
    frames = []
    for fpath in csv_files:
        label = extract_label_from_filename(fpath)
        if label == -1:
            print(f'WARNING: unknown label in {fpath}, skipping.')
            continue
        df = pd.read_csv(fpath, header=0, names=['timestamp', 'value'])
        df['label'] = label
        frames.append(df)
    all_data = pd.concat(frames, ignore_index=True)
    all_data.sort_values('timestamp', inplace=True)
    all_data.reset_index(drop=True, inplace=True)
    all_data.to_csv(f'ALL_DATA_{sensor_number}.csv', index=False)
    return all_data, f'ALL_DATA_{sensor_number}.csv'

def find_sensor_dir(base, sensor_num):
    for entry in os.listdir(base):
        if (f'Sensor{sensor_num}' in entry and 'Leakage' in entry
            and os.path.isdir(os.path.join(base, entry))):
            return os.path.join(base, entry)
    return base

BASE = EXTRACT_DIR
sensor1_dir = find_sensor_dir(BASE, 1)
all_data_1, f1 = build_all_data(sensor1_dir, 1)
sensor2_dir = find_sensor_dir(BASE, 2)
```

```
all_data_2, f2 = build_all_data(sensor2_dir, 2)
```

Appendix B — Temporal Aggregation and Sensor Merging

```
WINDOW = 0.1    # seconds per aggregation bucket
```

```
def aggregate_data(all_data_df, sensor_number):
    rows = all_data_df[['timestamp', 'value', 'label']].values
    aggr_rows = []
    bucket_start_ts = rows[0, 0]
    bucket_label = int(rows[0, 2])
    bucket_values = [rows[0, 1]]

    def flush_bucket(ts, vals, lbl):
        aggr_rows.append({
            'ts': ts,
            'average_value': float(np.mean(vals)),
            'min_value': float(np.min(vals)),
            'max_value': float(np.max(vals)),
            'label': lbl,
        })

    for i in range(1, len(rows)):
        ts = rows[i, 0]
        val = rows[i, 1]
        label = int(rows[i, 2])
        if label != bucket_label or (ts - bucket_start_ts) > WINDOW:
            flush_bucket(bucket_start_ts, bucket_values, bucket_label)
            bucket_start_ts = ts
            bucket_label = label
            bucket_values = [val]
        else:
            bucket_values.append(val)
    flush_bucket(bucket_start_ts, bucket_values, bucket_label)
    aggr_df = pd.DataFrame(aggr_rows,
        columns=['ts', 'average_value', 'min_value', 'max_value', 'label'])
    aggr_df.to_csv(f'AGGR_DATA_{sensor_number}.csv', index=False)
    return aggr_df

def merge_aggr(aggr1_df, aggr2_df):
    a1 = aggr1_df.copy().reset_index(drop=True)
    a2 = aggr2_df.copy().reset_index(drop=True)
    a2_ts = a2['ts'].values
    a2_end = a2_ts + WINDOW
    merged_rows = []
    used_s2 = set()

    def find_overlapping_s2(ts):
        row_end = ts + WINDOW
        mask = (a2_ts < row_end) & (a2_end > ts)
```

```

        return np.where(mask)[0]

    for _, r1 in a1.iterrows():
        ts = r1['ts']
        s2_idx = find_overlapping_s2(ts)
        if len(s2_idx) == 0:
            merged_rows.append({'ts': ts,
                                's1_avg': r1['average_value'], 's1_min': r1['min_value'],
                                's1_max': r1['max_value'],
                                's2_avg': np.nan, 's2_min': np.nan, 's2_max': np.nan,
                                'label': r1['label']})
        else:
            s2_sub = a2.iloc[s2_idx]
            merged_rows.append({'ts': ts,
                                's1_avg': r1['average_value'], 's1_min': r1['min_value'],
                                's1_max': r1['max_value'],
                                's2_avg': s2_sub['average_value'].mean(),
                                's2_min': s2_sub['min_value'].min(),
                                's2_max': s2_sub['max_value'].max(),
                                'label': r1['label']})
            used_s2.update(s2_idx.tolist())
    for i, r2 in a2.iterrows():
        if i in used_s2: continue
        merged_rows.append({'ts': r2['ts'],
                            's1_avg': np.nan, 's1_min': np.nan, 's1_max': np.nan,
                            's2_avg': r2['average_value'], 's2_min': r2['min_value'],
                            's2_max': r2['max_value'], 'label': r2['label']})
    merged_df = pd.DataFrame(merged_rows,

    columns=['ts', 's1_avg', 's1_min', 's1_max', 's2_avg', 's2_min', 's2_max', 'label'])
    merged_df.sort_values('ts', inplace=True)
    merged_df.reset_index(drop=True, inplace=True)
    merged_df.to_csv('AGGR_DATA_ALL.csv', index=False)
    return merged_df

aggr1 = aggregate_data(all_data_1, 1)
aggr2 = aggregate_data(all_data_2, 2)
merged_df = merge_aggr(aggr1, aggr2)

```

Appendix C — Preprocessing and Baseline Models

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.neural_network import MLPClassifier
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv('AGGR_DATA_ALL.csv')
X_raw = df.drop(columns=['ts', 'label']).values
y = df['label'].values.astype(int)

```

```

imputer = SimpleImputer(strategy='mean')
X_imp   = imputer.fit_transform(X_raw)
scaler  = StandardScaler()
X_sc     = scaler.fit_transform(X_imp)

X_train, X_temp, y_train, y_temp = train_test_split(
    X_sc, y, test_size=0.60, random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp)

# Model 1 - Shallow MLP
mlp = MLPClassifier(hidden_layer_sizes=(10,), activation='relu',
    solver='adam', learning_rate_init=0.001, max_iter=500, random_state=42)
mlp.fit(X_train, y_train)
y_val_pred = mlp.predict(X_val)
y_test_pred = mlp.predict(X_test)
print(f'Val Acc: {accuracy_score(y_val, y_val_pred):.4f}')
print(f'Test Acc: {accuracy_score(y_test, y_test_pred):.4f}')
print(classification_report(y_test, y_test_pred,
    target_names=['AllTapsClosed', 'Tap1Open', 'Tap2Open', 'Tap3Open', 'Tap4Open']))

# Model 2 - Decision Tree
X_eval = np.vstack([X_val, X_test])
y_eval = np.concatenate([y_val, y_test])
dt = DecisionTreeClassifier(random_state=42, max_depth=None)
dt.fit(X_train, y_train)
y_eval_pred = dt.predict(X_eval)
print(f'DT Eval Acc: {accuracy_score(y_eval, y_eval_pred):.4f}')
print(f'Tree depth: {dt.get_depth()} Leaves: {dt.get_n_leaves()}')

```

Appendix D — Feature Augmentation and Advanced Models

```

# — Feature engineering —————
df_aug = pd.read_csv('AGGR_DATA_ALL.csv')
LOOKBACK = 10.0
ts_vals = df_aug['ts'].values
s1_avg_vals = df_aug['s1_avg'].fillna(0).values
s2_avg_vals = df_aug['s2_avg'].fillna(0).values

max_abs_10s_s1 = np.zeros(len(df_aug))
max_abs_10s_s2 = np.zeros(len(df_aug))
for i in range(len(df_aug)):
    window_mask = (ts_vals >= ts_vals[i] - LOOKBACK) & (ts_vals <= ts_vals[i])
    max_abs_10s_s1[i] = np.max(np.abs(s1_avg_vals[window_mask]))
    max_abs_10s_s2[i] = np.max(np.abs(s2_avg_vals[window_mask]))

df_aug['max_abs_10s_s1'] = max_abs_10s_s1
df_aug['max_abs_10s_s2'] = max_abs_10s_s2
df_aug['s1_range'] = df_aug['s1_max'].fillna(0) - df_aug['s1_min'].fillna(0)

```

```
df_aug['s2_range'] = df_aug['s2_max'].fillna(0) - df_aug['s2_min'].fillna(0)
df_aug.to_csv('AGGR_DATA_ALL_AUG.csv', index=False)
```

```
# — Preprocessing —————
```

```
X_aug_raw = df_aug.drop(columns=['ts', 'label']).values
y_aug      = df_aug['label'].values.astype(int)
imp2       = SimpleImputer(strategy='mean')
X_aug_i    = imp2.fit_transform(X_aug_raw)
sc2        = StandardScaler()
X_aug_s    = sc2.fit_transform(X_aug_i)
```

```
Xtr, Xtmp, ytr, ytmp = train_test_split(X_aug_s, y_aug,
                                         test_size=0.60, random_state=42, stratify=y_aug)
Xv, Xte, yv, yte = train_test_split(Xtmp, ytmp,
                                     test_size=0.50, random_state=42, stratify=ytmp)
```

```
# — Model 3 - Deep MLP —————
```

```
mlp_deep = MLPClassifier(hidden_layer_sizes=(64, 32, 16),
                          activation='tanh', solver='adam', learning_rate='adaptive',
                          max_iter=1000, random_state=42, alpha=0.01)
mlp_deep.fit(Xtr, ytr)
deep_acc = accuracy_score(yte, mlp_deep.predict(Xte))
deep_f1 = f1_score(yte, mlp_deep.predict(Xte), average='weighted')
print(f'Deep MLP Accuracy: {deep_acc:.4f}    F1: {deep_f1:.4f}')
```

```
# — Model 4 - CNN —————
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
tf.random.set_seed(42)
```

```
n_features = Xtr.shape[1]
Xtr_cnn = Xtr.reshape(-1, n_features, 1)
Xv_cnn = Xv.reshape(-1, n_features, 1)
Xte_cnn = Xte.reshape(-1, n_features, 1)
n_classes = 5
ytr_oh = keras.utils.to_categorical(ytr, n_classes)
yv_oh = keras.utils.to_categorical(yv, n_classes)
yte_oh = keras.utils.to_categorical(yte, n_classes)
```

```
cnn_model = keras.Sequential([
    layers.Input(shape=(n_features, 1)),
    layers.Conv1D(32, kernel_size=3, activation='relu', padding='same'),
    layers.Conv1D(64, kernel_size=3, activation='relu', padding='same'),
    layers.GlobalMaxPooling1D(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(n_classes, activation='softmax'),
])
cnn_model.compile(optimizer='adam',
                  loss='categorical_crossentropy', metrics=['accuracy'])
```

```

early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=10, restore_best_weights=True)
history = cnn_model.fit(Xtr_cnn, ytr_oh,
    validation_data=(Xv_cnn, yv_oh),
    epochs=100, batch_size=32, callbacks=[early_stop], verbose=1)

cnn_loss, cnn_acc = cnn_model.evaluate(Xte_cnn, yte_oh, verbose=0)
y_cnn_pred = np.argmax(cnn_model.predict(Xte_cnn), axis=1)
cnn_f1 = f1_score(yte, y_cnn_pred, average='weighted')
print(f'CNN Accuracy: {cnn_acc:.4f}    F1: {cnn_f1:.4f}')

# — Training curves —————
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history.history['accuracy'], label='Train')
axes[0].plot(history.history['val_accuracy'], label='Val')
axes[0].set_title('CNN Accuracy'); axes[0].legend()
axes[1].plot(history.history['loss'], label='Train')
axes[1].plot(history.history['val_loss'], label='Val')
axes[1].set_title('CNN Loss'); axes[1].legend()
plt.tight_layout()
plt.savefig('cnn_training_curves.png', dpi=150)
plt.show()

```